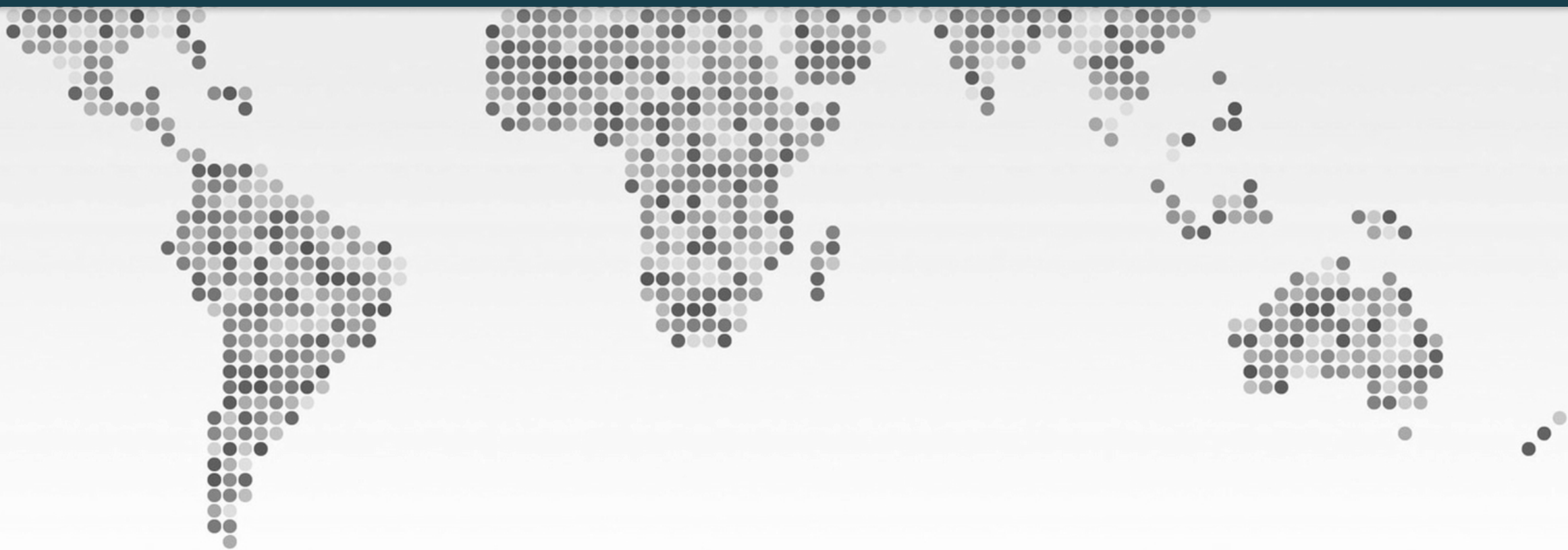


Software Engineering for Artificial Intelligence

AI-Enabled Systems in Operation



Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

After architecting and deployment, we have an AI-enabled system ready to be receiving external inputs and making predictions. This phase is called *operation*.

Operation. The tasks, infrastructure, and processes involved in deploying, running, and updating a software system and the hardware it runs on in production.

Operations is often considered as a **distinct activity** from the development of the software and is often performed by other team members with dedicated expertise.

DevOps and **MLOps** emerged as concepts and infrastructure to support collaboration between developers, data scientists, and operators toward better operations.

Compared to traditional software, introducing AI raises additional challenges during operations, such as (a) ensuring that model training and model inference operate well and (b) moving and processing very large amounts of data.

In addition, AI-enabled systems often have demanding requirements for updates and experimentation due to continuous experimentation and drifting data.

When software projects with AI components reach a certain scale or complexity, organizations should consider bringing in operations specialists, who may have role titles such as **system administrators**, **release engineers**, **site reliability engineers**, **DevOps engineers**, **MLOps engineers**, **data engineers**, and **AI engineers**.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

Since operation benefits from certain design qualities of the software system, preparing the system for future operations is another important design task that should be considered early on in a system's development, just as the other design and architecture considerations discussed before.

The operations team will have a much easier time when developers build **observability** into the system's infrastructure early on and **virtualize** the software for easy deployment.

Conversely, designing the system for operations allows the operators to focus on providing **value back to developers** by causing fewer interruptions and enabling **faster experimentation**.

In this sense, it is useful to characterize the developers involved in the process based on the expertise they have.



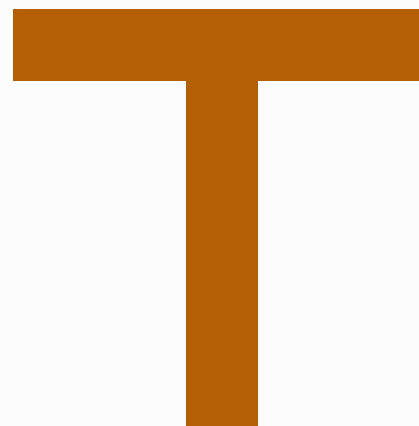
I-Shaped

Deep expertise in one topic



Generalist

Broad knowledge of many topics,
but not expert in any



T-Shaped

Expert in one topic and broad
knowledge of other topics

T-Shaped team members combine deep expertise in one topic with broad knowledge of others. They are **ideal members** in interdisciplinary teams, since they can effectively understand and collaborate with others.

AI-Enabled Systems in Operation

Following the T-shaped team members' argument, software engineers, data scientists, managers, and other team members benefit from understanding the **key concerns of operators** that allows them to anticipate their requirements, to involve them early on, and to design the system to be more operations-friendly.

As usual, it is a good idea to start with **requirements for operations**: these are typically called *service-level objectives*. Typical service level objectives include:

- 1 **Minimum response latency**: The longest time a system takes to respond to an input under worst-case conditions, i.e., processing delays, network latency, and resources.
- 2 **Minimum system throughput**: The lowest rate at which a system successfully processes and outputs tasks, requests, or data over a given time period, typically measured under worst-case conditions, such as high load or resource constraints.
- 3 **Minimum availability**: The lowest guaranteed percentage of time a system remains operational and accessible over a specified period, considering planned maintenance and potential failures.
- 4 **Maximum error rate**: The highest allowable proportion of errors in a system's output relative to the total number of operations, requests, or transactions over a given period.
- 5 **Time to deploy an update**: The total duration required to release a system update, including code integration, testing, packaging, deployment, and validation in production.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

Following the T-shaped team members' argument, software engineers, data scientists, managers, and other team members benefit from understanding the **key concerns of operators** that allows them to anticipate their requirements, to involve them early on, and to design the system to be more operations-friendly.

As usual, it is a good idea to start with **requirements for operations**: these are typically called *service-level objectives*.

Operators **can usually not achieve these goals alone** but depend on the capabilities of the software they deploy and operate. For instance, ensuring low response latency requires not only optimized infrastructure but also efficient application code and well-designed data access patterns. Similarly, maintaining high availability depends on both resilient system architecture and robust failure recovery mechanisms.

Hence, collaboration between operators, software engineers, and data scientists is essential to deploy systems that achieve their service level objectives in production.

Service level objectives can and should be defined for the **system as a whole**, as well as for **individually operated components**.

Service quality measures of system and component **may be** related, but do not have to be—for example, error-handling mechanisms in the user-facing components can preserve system operation even when individual components fail in the background.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

Service level objectives are negotiated and renegotiated between operators and other stakeholders, reasoning about constraints, priorities, and trade-offs as usual.

For example, the increased infrastructure cost to increase availability from 99.9 to 99.99 percent might simply outweigh the benefit; even lower availability might be acceptable for a service if it allows the operation team to focus on more important goals.

In more formal arrangements, service-level objectives may be codified as contracts called **service-level agreements**, which typically include a description of consequences and penalties if the objectives are not met.

Service-level agreement. This is a formal contract between a *service provider* and a *client* that defines the *expected level of service*, including key performance metrics such as availability, response time, throughput, and error rates. It also *specifies responsibilities, penalties for non-compliance, and any compensation mechanisms* in case of service failures.

Nowadays, SLAs are *widely used across industries* to ensure service reliability, maintain accountability, and enhance customer satisfaction in both *business-to-business (B2B)* and *business-to-customer (B2C)* interactions.

For example, consider when you subscribe to a mobile phone plan: you formally agree the terms of service and the contract is the basis for the mutual relationship.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Observability

Being responsible for keeping systems running and performing well with changing demands, operators need visibility into how the system is doing. This is often described as **monitoring** the system and is broadly captured by the term **observability**.

Observability. The ability to *monitor, analyze, and understand* the *internal state and behavior* of an AI system based on its external outputs, logs, metrics, and traces.

Observability goes beyond traditional monitoring by providing insights into model performance, decision-making processes, and system health to ensure reliability, fairness, and compliance to the service-level agreement.

Key aspects of observability in AI-enabled systems include:

- 1 **Model Performance Monitoring:** Tracking accuracy, precision, recall, and drift in AI models.
- 2 **Data Quality and Integrity:** Ensuring input data is consistent, unbiased, and up-to-date.
- 3 **Explainability & Transparency:** Providing insights into how AI decisions are made.
- 4 **Bias and Fairness Analysis:** Detecting unintended discrimination or bias in predictions.
- 5 **System Reliability & Latency:** Monitoring response times and uptime for operational stability.
- 6 **Error Detection & Root Cause Analysis:** Identifying anomalies, failures, or unintended behavior.

AI-Enabled Systems in Operation: The Three Pillars of Observability

Going deeper, we can think at observability as a *window* into our system through which we can observe how things are going on and how we should act on the system itself to keep it working well.

This is because AI-intensive systems must live in a changing environment. As such, these systems inherit all the maintenance and evolution challenges that you might have heard already in other courses.

SOFTWARE MAINTENANCE AND EVOLUTION INTERMEZZO - THE LEHMAN'S LAWS

Continuous changes. A system must be continually adapted or it becomes progressively less satisfactory.

Increasing complexity. As a system evolves, its complexity increases unless work is done to maintain or reduce it.

In AI-intensive terms, this implies that machine learning models need to be periodically retrained to maintain performance. However, **you cannot control what you cannot measure**: observability is therefore a key concern in real-world AI.

To enable observability in practice, we typically refer to three pillars, namely **metrics, logs, and traces**.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability

Metrics. Numerical measurements that capture the behavior or performance of a system over time. They are structured, aggregatable, and ideal for real-time monitoring and alerting.

Metrics are important in many ways. At first, they allow **tracking** how the system is behaving right now. Also, they may be used to **automatically detect anomalies, slowdowns, or outages**; or even to enable **performance benchmarking** by comparing system behavior across versions. From a managerial perspective, they provide **measurable evidence** that service-level objectives are being met.

More importantly, metrics allow the detection of **silent failures**, like a model that starts making biased or irrelevant predictions due to environmental changes—issues that won't necessarily appear in other observability instruments like logs or exceptions.

General system metrics may include CPU/GPU utilization, memory usage, request latency, throughput, error rate, availability/uptime, or queue size.

AI-specific metrics may provide additional information useful to **schedule retraining** and **ease debugging**. For instance, model accuracy, inference latency, prediction confidence and model usage frequency. Additionally, metrics can be tailored to identify *prediction and feature drifts*.

AI-Enabled Systems in Operation: The Three Pillars of Observability

Metrics. Numerical measurements that capture the behavior or performance of a system over time. They are structured, aggregatable, and ideal for real-time monitoring and alerting.

1 Prediction drift. Change in the distribution of predicted outputs over time.

In this case, statistics come into play. A popular metric is called *Kullback-Leibler Divergence*, which measures how one probability distribution diverges from another. Suppose that the probability distribution on the training set is the following: {Class A: 40%, B: 30%, C: 30%}. If, in the last week, the probability distribution of the model is: {A: 20%, B: 40%, C: 40%}, this may indicate a potential drift.

Among the most common causes:

- *Input Data Drift (Covariate Shift)*: The distribution of input features changes, affecting model outputs. For instance, a recommendation system sees new user behavior patterns (e.g., after a major event or holiday).
- *Concept Drift*: The relationship between inputs and outputs changes over time. For instance, a spam detection model trained in 2021 fails to catch new spam patterns in 2025.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability

Metrics. Numerical measurements that capture the behavior or performance of a system over time. They are structured, aggregatable, and ideal for real-time monitoring and alerting.

2 **Feature drift.** Change in the distribution of input features compared to training.

Suppose that in the training data, the average value of the feature "age" was 35, with a standard deviation of 8. Now, after monitoring the last week's production data, we observe a mean of 47 and a standard deviation of 6. This shift in the central tendency (mean) and spread (standard deviation) suggests that the demographics of users interacting with the system have changed.

This does not necessarily mean the model is wrong — but it **may start making worse predictions**, especially if the new inputs fall outside the model's experience.

Similar statistical approaches can be used to monitor feature drift. This may be caused by changes in AI usage, e.g., user or system behavior evolves, or by environmental changes, e.g., the environment alters sensor data used to collect inputs.

In other words, monitoring feature drift may lead data scientists to early catch variations in the input data that may worsen model performance, anticipating retraining. Not all feature drift is *bad* — but unmonitored drift is dangerous.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability

Logs. Timestamped textual records of events, actions, or messages generated by a software system during its execution.

Logs are typically organized in *human-readable text*, though often semi-structured or JSON-formatted. They are *event-based*, namely that new text is generated only when something specific happens (e.g., an error, a user request, or a model response). They must be *actionable*, including timestamps to trace the sequence of events and metadata like user ID, request ID, feature values, model version, etc.

```
[2025-03-21 09:12:45] INFO: Received request for prediction – user_id=1243  
[2025-03-21 09:12:45] DEBUG: Input vector: [0.42, 1.12, 3.50]  
[2025-03-21 09:12:45] INFO: Model returned class=B (confidence=0.78)  
[2025-03-21 09:12:46] ERROR: Inference failed – missing required feature 'age'
```

A good use of logs may help pinpointing root causes of issues, analyzing usage patterns and detect anomalies, or understanding how the model behaves in production.

From a managerial perspective, they contribute to *auditability* and *compliance*, tracking who did what and when: if something goes wrong with the system or in case of legal concerns, logs may provide evidence useful to the debate.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability

Traces. They capture the end-to-end flow of a single request or transaction as it travels through the components of a distributed system.

In other terms, they represent a **request's journey** across multiple services or microservices: traces are composed of **spans**, each representing a specific operation (e.g., an HTTP call, a database query).

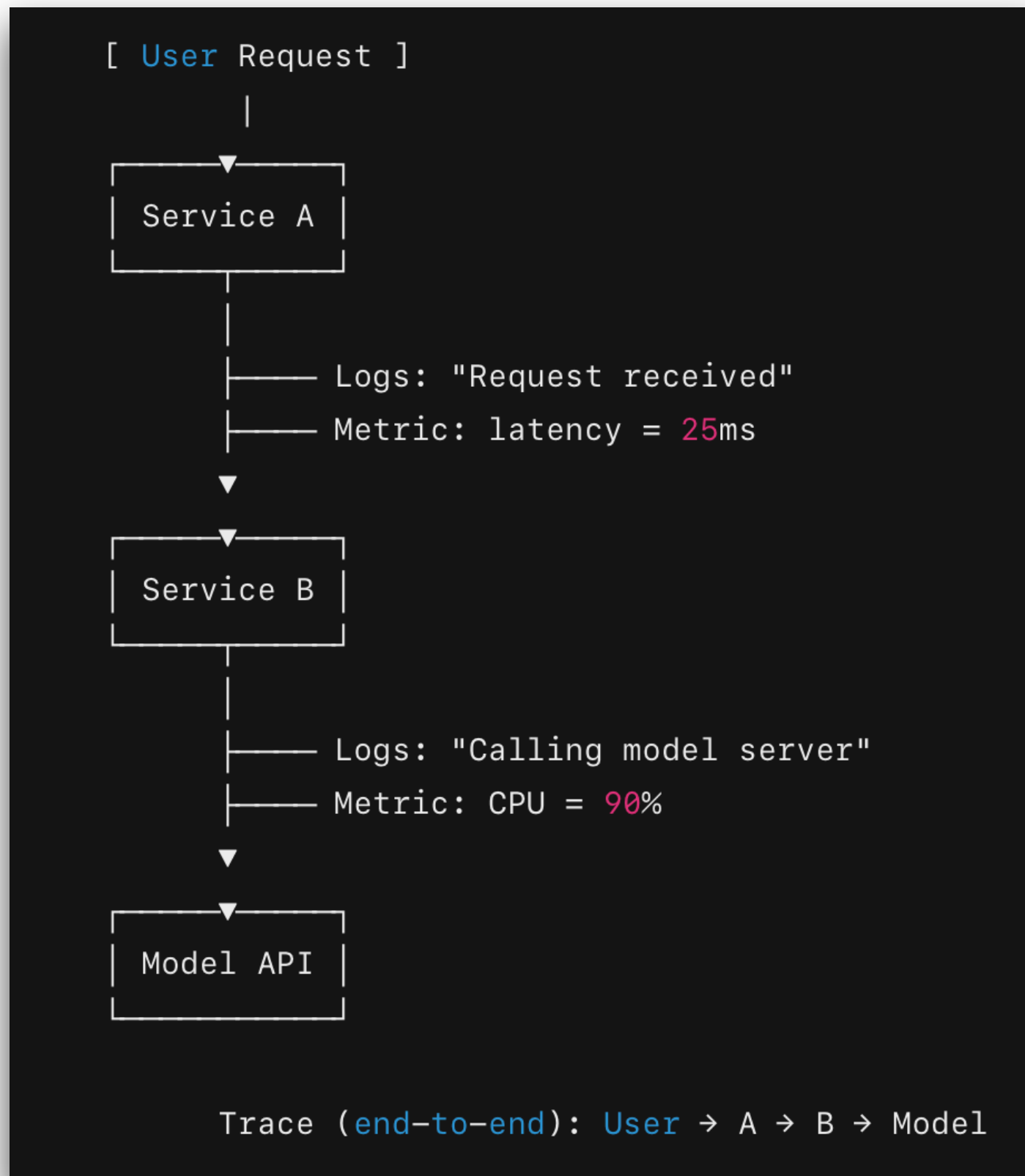
```
Trace ID: 789abc
- Span 1: [User Service] → Receive request (15ms)
- Span 2: [Recommendation API] → Call model server (40ms)
- Span 3: [Model Server] → Load features, run inference (60ms)
- Span 4: [Database] → Fetch additional metadata (25ms)
```

Traces may help identifying which services or components are slow within an AI-enabled system, possibly detecting bottlenecks. Also, they help understand how microservices interact, possibly suggesting opportunities for refactoring.

While **logs** tell you *what happened*, and **metrics** tell you *how often* it happens, **traces** tell you *where it happened across your system* — they are especially powerful in microservice-based AI systems.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability



Altogether, metrics, logs, and traces offer an instrument to continuously monitor and analyze how services work and how they interact.

Model retraining, model rollback, and alerting are decisions typically taken by analyzing observability data, such as feature and prediction drift, degradation in model accuracy or confidence, and system-level metrics like latency and error rates.

At the same time, observability data allow auditing the behavior of AI-enabled systems over time, helping to ensure compliance with regulations, detect unexpected behavior, and trace back the causes of incorrect/unfair predictions.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Observability & Telemetry

Typically, observing a system consists of three distinct steps: *instrumenting the system to produce telemetry signals* that can be monitored, *collecting and storing the telemetry signals*, and *analyzing the telemetry signals*.

Producing telemetry with instrumentation. In the simplest case, the application code simply includes print statements to write information into log files. Various libraries for software metrics and telemetry provide APIs to record counters and distributions within the application code.

Much infrastructure used in modern software systems already logs many events that can provide a source of telemetry. For example, web servers will log many events out of the box that may provide useful insights, such as logging every single request.

Collecting telemetry. Many infrastructure projects have been developed to collect and store such signals in a unified place for subsequent analysis. For example, *Prometheus* regularly requests updated metrics from all configured systems.

The telemetry-collection phase focuses on moving large amounts of telemetry data efficiently and with low latency from many sources to a single place where telemetry can be analyzed.

Analyzing telemetry. Many different forms of analyses can be performed on the collected signals. Typically, information is extracted and aggregated from raw telemetry data, such as computing the average response time from a server log.

The most common forms of analysis are real-time trends of measures shown in dashboards and automated alerts that are triggered when the observed values exceed predefined thresholds.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

Observability is strictly connected to the concept of automating the deployment of AI-enabled systems. Indeed, most software systems must be updated frequently to fix bugs and to deploy new features. Long gone are the days where a new software version is released every three years; today, many organizations release and deploy updates **weekly** or even **multiple times a day**.

Deploying software and software updates is a **core responsibility of operators**, and deployment is often a source of frustration. Unscheduled patches to fix an urgent problem live in production are an operator's nightmare.

Operators often invest substantial effort in automating deployments. By eliminating manual steps, they avoid mistakes and inconsistencies, increase the speed with which updates can be deployed, and support rolling back problematic updates.

Most organizations start automation with a *continuous integration* service.

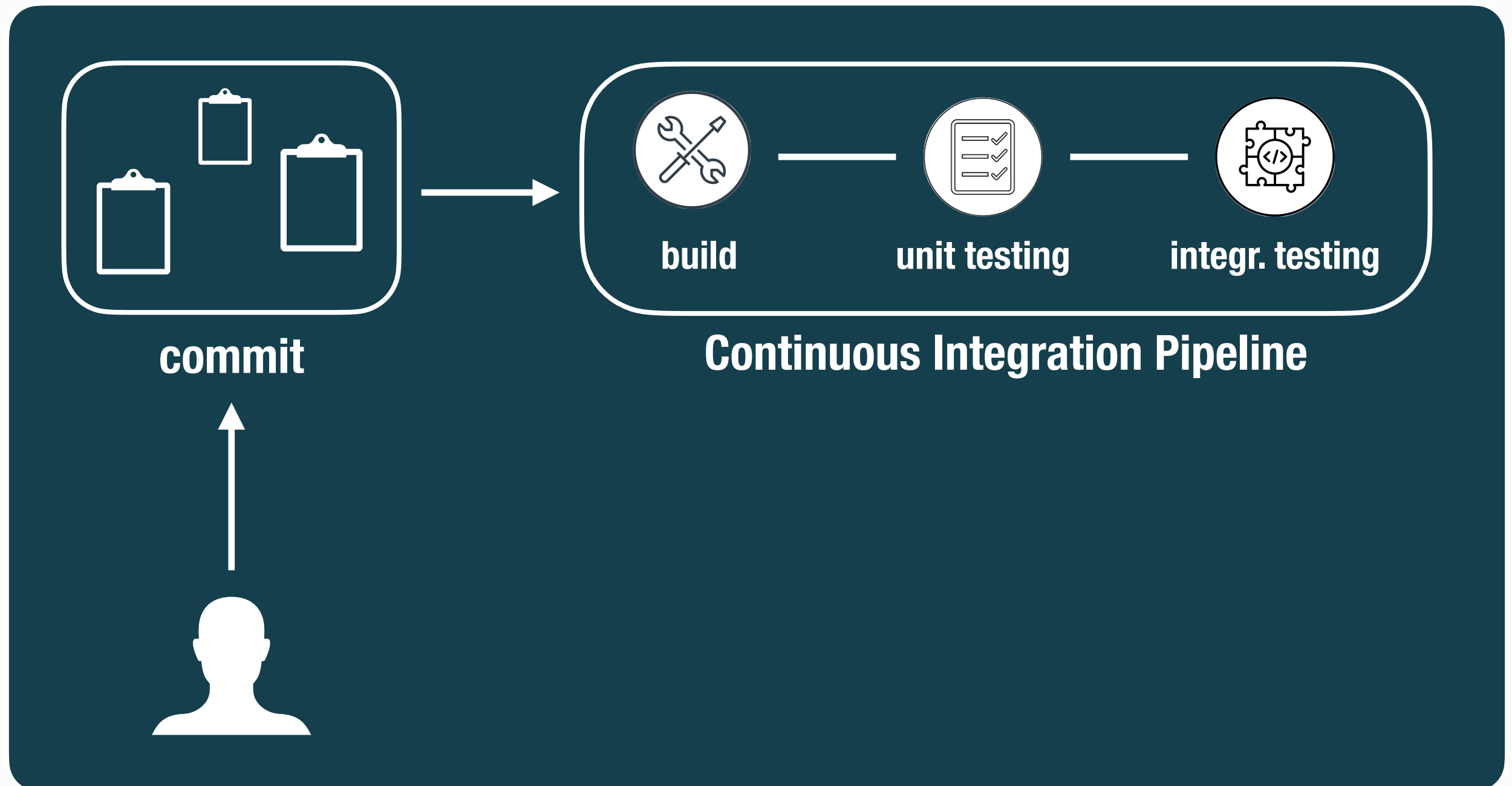
Continuous Integration. The process of automatically building and testing every code change integrated into the version control system.

Continuous integration ensures that each version can be built independently (not just on the developer's machine), pass all tests, and pass all other automated quality checks (e.g., static analysis).

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

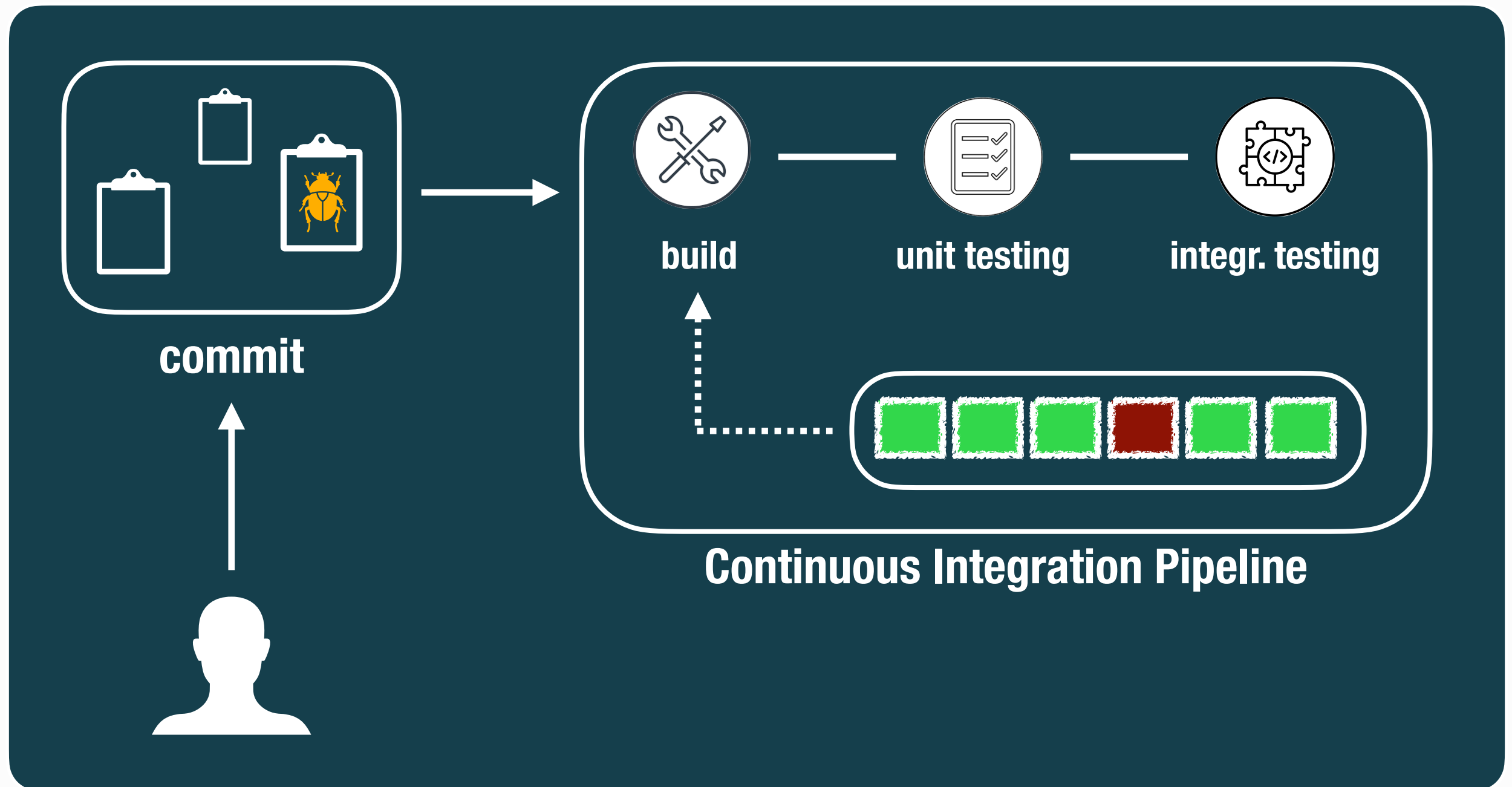
Continuous Integration. The process of automatically building and testing every code change integrated into the version control system.



Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

Continuous Integration. The process of automatically building and testing every code change integrated into the version control system.



Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

Continuous Integration. The process of automatically building and testing every code change integrated into the version control system.

Typically, a dedicated machine will automatically download the source from the version control system after every change, run the build scripts, and then execute all tests, reporting found problems back to developers.

This way, it is not possible to “forget” to include a dependency or “forget” to execute tests before a commit, even when in a rush.

Nowadays, continuous integration represents a key enabler of quality assurance in modern software systems. By automatically building and testing code every time a change is committed, continuous integration *helps detect errors early* and *reduce integration problems*, which is vital for systems having continuous updates.

CI pipelines typically include automated tests, static code analysis, and code style checks, which act as first-line defenses against regressions and code quality issues. We will discuss various forms of automated quality assurance throughout the quality assurance and responsible AI chapters.

More importantly for the scope of this lecture, CI **ensures that software remains in a deployable state throughout the development lifecycle.**

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

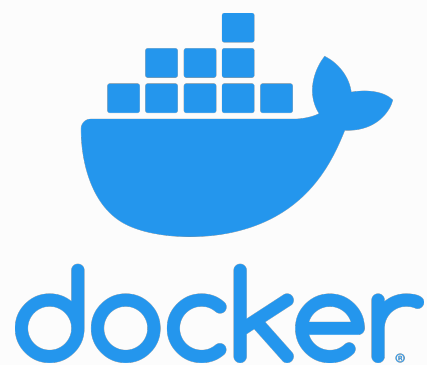
AI-Enabled Systems in Operation: Automating Deployments

Integrating CI with continuous delivery/deployment (CD) systems allows teams to automate deployment processes and validate software in production-like environments, enhancing reliability and reducing time-to-market.

Continuous Delivery. The process of automatically staging code changes for release, ensuring that software can be deployed to production at any time with minimal manual intervention.

In the simplest case, handwritten scripts upload binaries or models to a server, but often more sophisticated steps are involved, e.g., packaging the artifacts in a *container*, uploading the container to a cloud service, launching one or more instances of the container, adjusting network routing, and then shutting down the previous containers.

A **container** is a lightweight, portable, software package that includes everything needed to run an application—such as code, runtime, system tools, libraries, and settings—ensuring consistent execution across different computing environments.



Docker is a well-known example: it allows developers to package applications and their dependencies into standardized containers, which can run reliably on any environment that supports the Docker engine—whether it's a developer's laptop, a testing server, or a production system in the cloud.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments and Virtualization

Why **containerization**? Installing software with many dependencies and configuring network routing on a new machine can be challenging. In the past, operators have often complained that they failed to install a release that worked fine on a developer's machine, due to *unnoticed* and *undocumented dependencies* or *system configs*.

It is even harder to ensure that hundreds or thousands of machines in a cluster or cloud deployment have *consistent configurations*. Ideally, we want to ensure that every single instance of a service has the **same version of the code**, the **same version of the model**, and the **same version of all libraries**, but also **the same configuration of service accounts, file permissions, network services, and firewalls**.

Virtualization technologies have significantly helped with **reducing friction between developers and operators** —> reducing socio-technical issues! Instead of operators deploying a binary to many machines and setting system configurations on each machine, developers now *routinely set up, configure, and test their code in a container*.

In this process, they essentially install their code on a minimal fresh operating system and would notice immediately missing dependencies or incompatible versions.

Also, system configurations like file permissions are set only once when creating the container. The operator then can take this container as a single large binary artifact and push copies to the machines that will run it, focusing on *hardware allocation* and *networking* rather than on installing libraries and modifying local configuration files.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments and Virtualization

All machines that execute a container share the same consistent environment within the container. Furthermore, the code in the container is entirely separated from other code that may be run on the same machine. Also rolling back a release is as easy as switching back to the previous container.

So, suppose to have our microservice implementing the *model inference service*. For example, a simple Python program can load the model and implement the model inference function, with or without feature encoding code; it can then use the library *flask* to accept HTTP requests on a given port, and for each request run the inference function and return the result as the HTTP response.

```
from flask import Flask, request, jsonify
app = Flask(__name__)
translation_model = ... # load model
@app.route('/translate', methods=['POST'])
def pred():
    text_to_translate = request.json.get('text')
    target_lang = request.json.get('target')
    output_array = translation_model(text_to_translate, target_lang)
    return jsonify({"response": decode(output_array)})
```

We want this code to be included within a virtual machine that can be ported to any hardware without the need of manual intervention

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments and Virtualization

All machines that execute a container share the same consistent environment within the container. Furthermore, the code in the container is entirely separated from other code that may be run on the same machine. Also rolling back a release is as easy as switching back to the previous container.

So, suppose to have our microservice implementing the *model inference service*. For example, a simple Python program can load the model and implement the model inference function, with or without feature encoding code; it can then use the library *flask* to accept HTTP requests on a given port, and for each request run the inference function and return the result as the HTTP response.

```
request, jsonify\n\n@model.route('/inference', methods=['POST'])\ndef inference(request):\n    text = request.json.get('text')\n    target = request.json.get('target')\n    output_array = model.predict(text_to_translate, target=target)\n    return jsonify({'output_array': decode(output_array)})
```

```
FROM python:3.8-buster\nRUN pip install uwsgi==2.0.20\nRUN pip install numpy==1.22.0\nRUN pip install tensorflow==2.7.0\nRUN pip install flask==2.0.2\nRUN pip install gunicorn==20.1.0\nCOPY models/model.pf /model/\nCOPY ./serve.py /app/main.py\nWORKDIR /app\nEXPOSE 4040\nCMD ["gunicorn", "-b 0.0.0.0:4040", "main:app"]
```

A Dockerfile has a similar structure: it creates a virtual machine for our model and flask-based inference server, specifying the libraries that should be included.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments and Virtualization

All machines that execute a container share the same consistent environment within the container. Furthermore, the code in the container is entirely separated from other code that may be run on the same machine. Also rolling back a release is as easy as switching back to the previous container.

So, suppose to have our microservice implementing the *model inference service*. For example, a simple Python program can load the model and implement the model inference function, with or without feature encoding code; it can then use the library *flask* to accept HTTP requests on a given port, and for each request run the inference function and return the result as the HTTP response.

A Dockerfile of this type explicitly specifies which version of Python to use, which dependencies to install in which version, which code and model files to deploy, how to start the code, and what network port to expose. **The operator does not need to figure out any of these tasks or manage version conflicts** because other language or library versions may already be installed on the system.

Just as the application code and individual components are virtualized with containers, it is also common to virtualize other infrastructure components. For instance, in case our application includes a MongoDB database holding the content to be analyzed by the AI component, we may want to include the database in our virtual environment, specifying the characteristics and libraries that enable its use.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

While virtualization guarantees that operators should not worry about the underlying code requirements, they still require setting up and managing virtual machines, including operating systems, resource allocation, and updates.

To avoid repetitive manual system configuration work and potential inconsistencies from manual changes, there is a strong push toward automating configuration changes.

That is, rather than remotely accessing a system to install a binary and adding a line to a configuration file, an operator will write a script to perform these changes and then run the script on the target machine.

Please notice that even if the configuration is applied only to a single machine, having a script will enable setting up the machine again in the future, without having to remember all manual steps done in the past. In addition, it is now possible to apply the same steps to many machines to configure them consistently, e.g., to ensure that they all run the same firewall configuration.

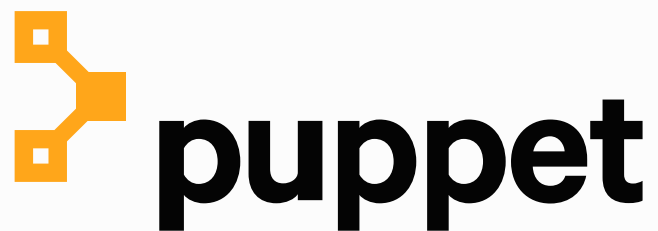
Infrastructure-as-code. The practice of managing and provisioning computing infrastructure through machine-readable configuration files, rather than manual processes, enabling automation, consistency, and repeatability.

The scripts and all configuration values they set are usually tracked in version control, just like any source code in the project.

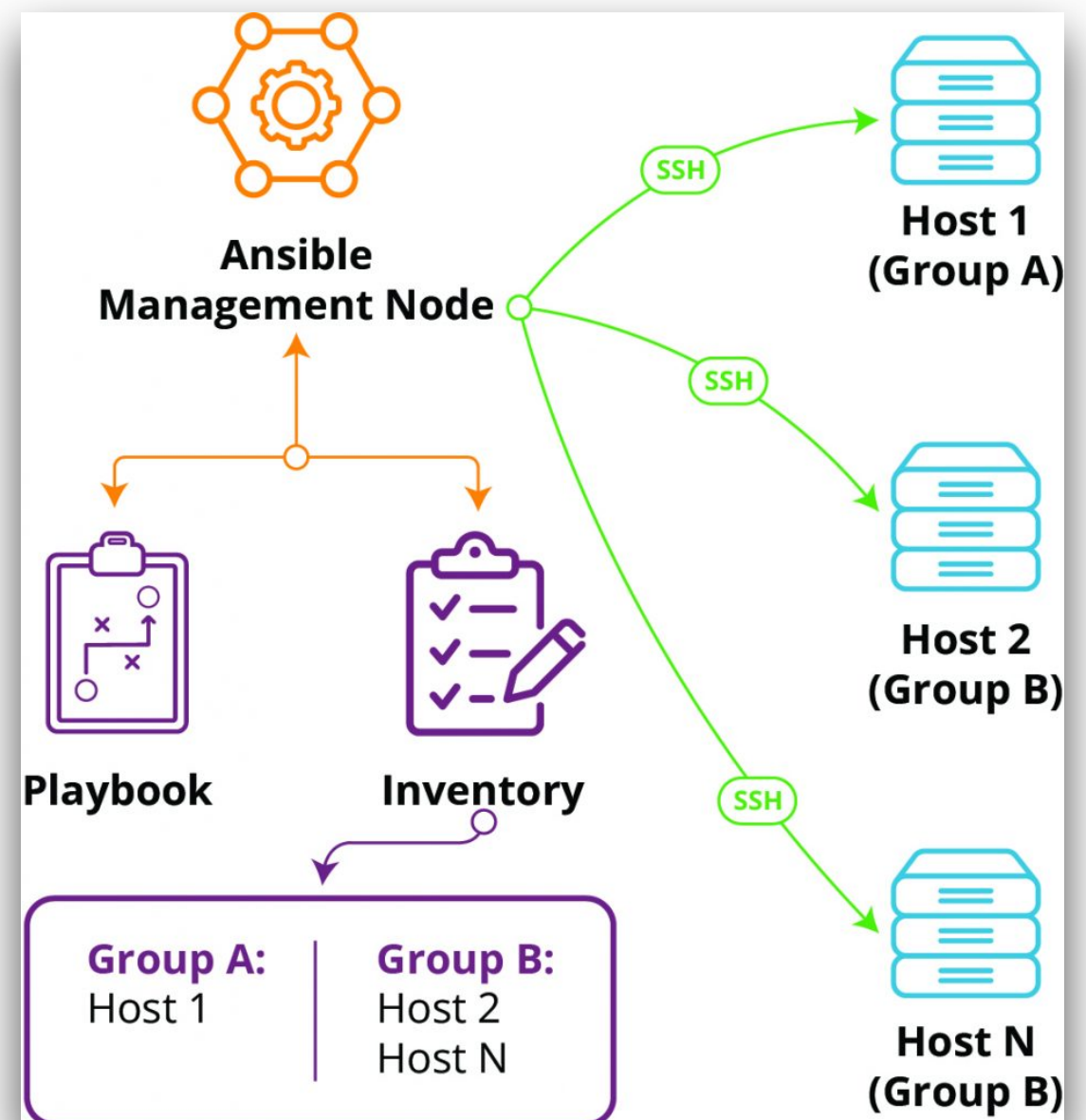
Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

With the increased adoption of containers, configuration scripts are usually less used for setting up applications, but more to set up the infrastructure in which the containers can be executed, to configure the network, and to install any remaining software that should not run within containers, such as tools recording hardware telemetry for system monitoring.



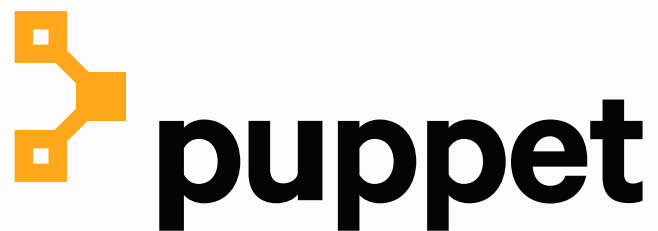
A **playbook** in Ansible is a YAML file (a human-readable data serialization language) that defines a set of tasks to be executed on remote hosts. It contains instructions for software provisioning, configuration management, and application deployment.



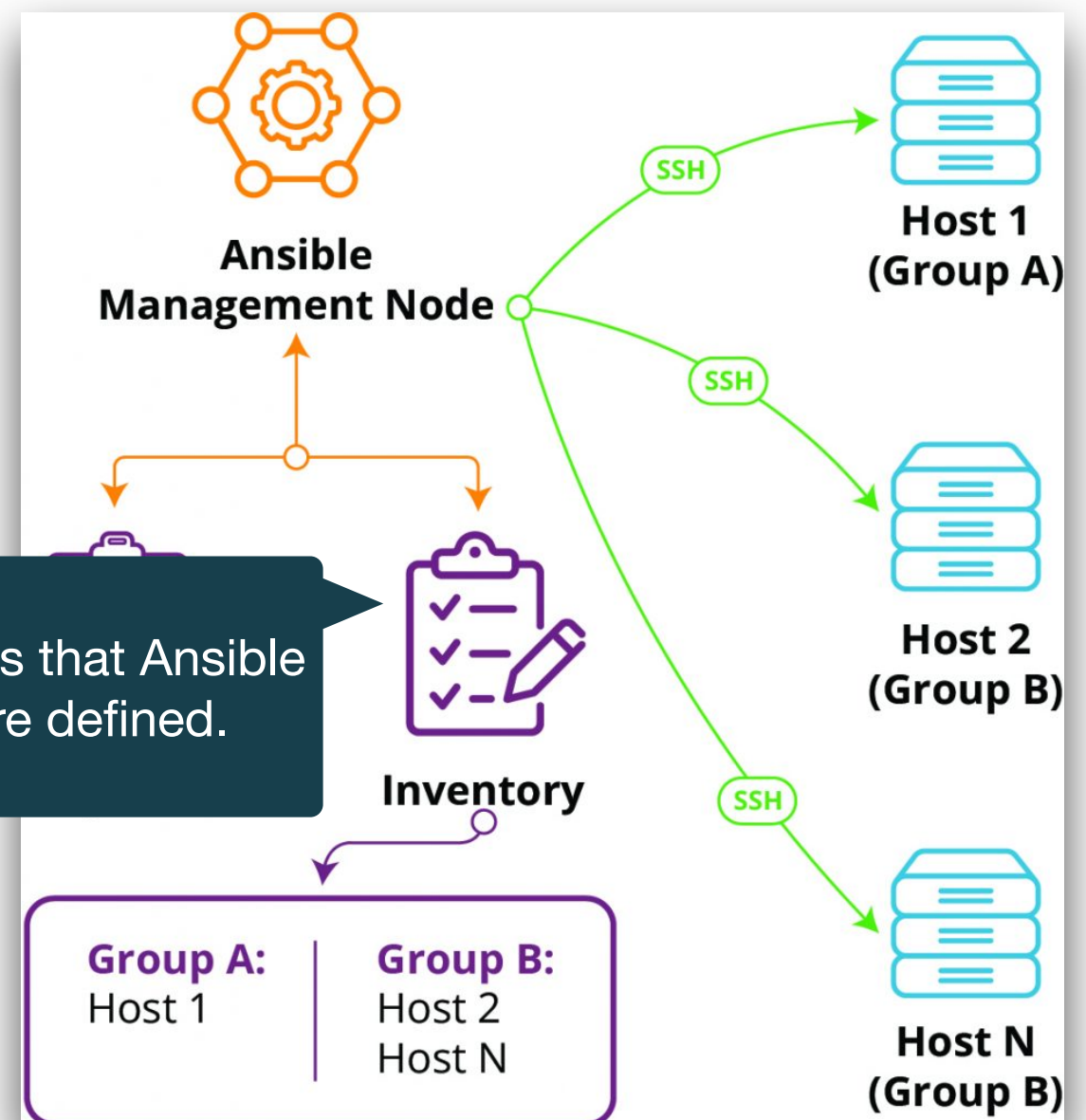
Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

With the increased adoption of containers, configuration scripts are usually less used for setting up applications, but more to set up the infrastructure in which the containers can be executed, to configure the network, and to install any remaining software that should not run within containers, such as tools recording hardware telemetry for system monitoring.



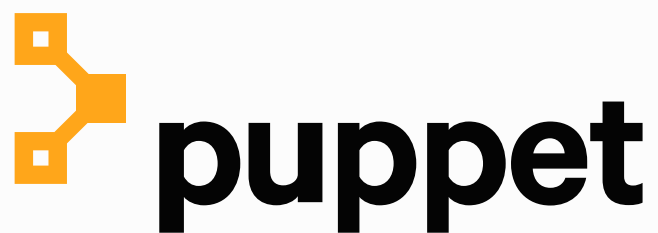
The **inventory** lists the hosts or groups of hosts that Ansible will manage. In this example, two groups are defined.



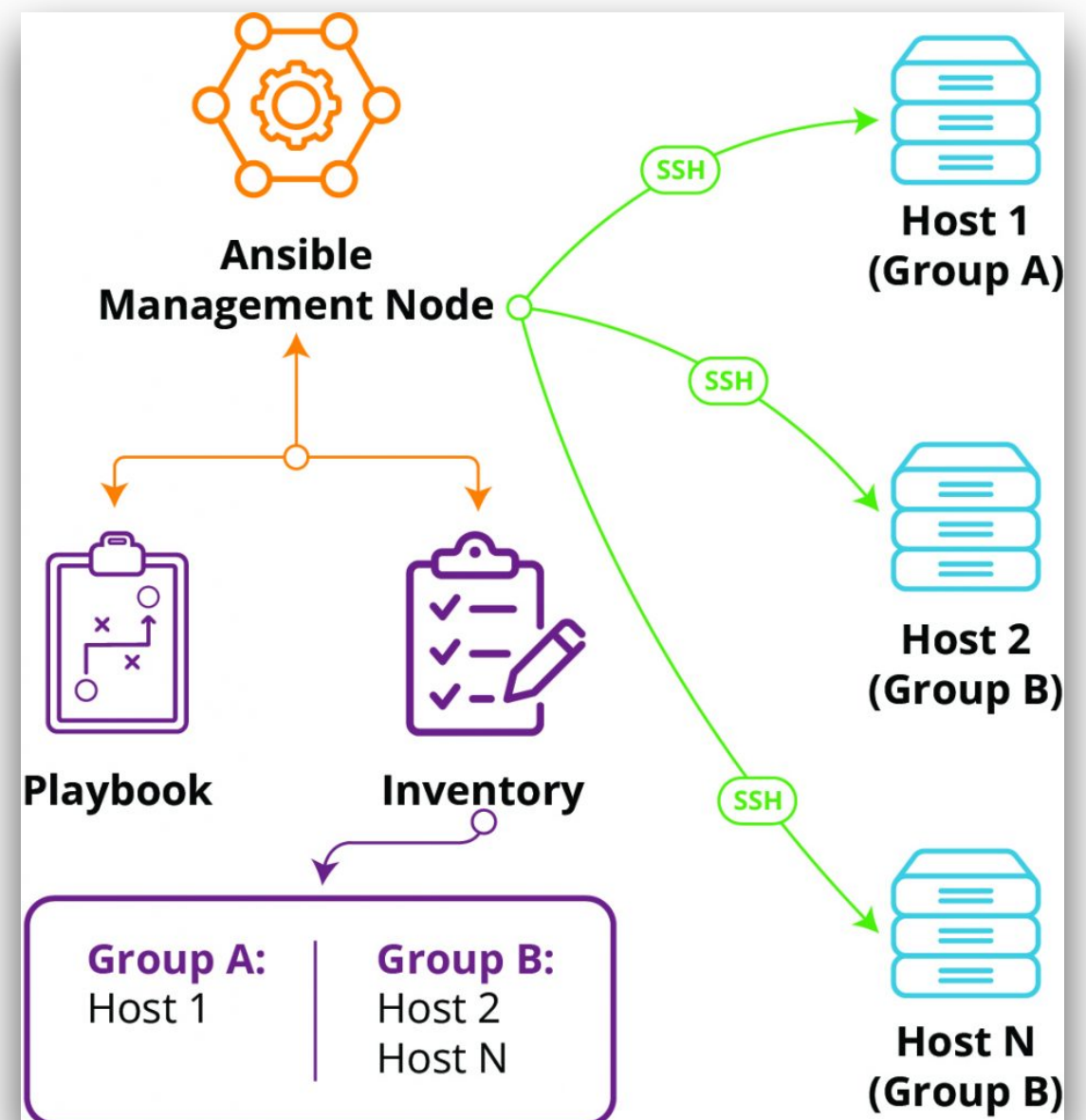
Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

With the increased adoption of containers, configuration scripts are usually less used for setting up applications, but more to set up the infrastructure in which the containers can be executed, to configure the network, and to install any remaining software that should not run within containers, such as tools recording hardware telemetry for system monitoring.



- 1 The **Ansible Manager** reads the **inventory** to determine which hosts it will configure.
- 2 The **playbook** defines the automation steps.
- 3 Ansible connects to remote **hosts** using **SSH**.
- 4 It executes the **playbook instructions** on the specified host groups.
- 5 The automation tasks (e.g., configuring settings) are completed on the hosts.



Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

With the increased adoption of containers, configuration scripts are usually less used for setting up applications, but more to set up the infrastructure in which the containers can be executed, to configure the network, and to install any remaining software that should not run within containers, such as tools recording hardware telemetry for system monitoring.

```
- hosts: all
  become: true
  tasks:
    - name: Install aptitude using apt
      apt: name=aptitude state=latest update_cache=yes force_apt
    - name: Install required system packages
      apt: name={{ item }} state=latest update_cache=yes
      loop: [ 'apt-transport-https', 'ca-certificates', 'curl' ]
    - name: Add Docker GPG apt Key
      apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
        state: present
    - name: Add Docker Repository
      apt_repository:
        repo: deb https://download.docker.com/linux/ubuntu bionic
        state: present
    - name: Update apt and install docker-ce
      apt: update_cache=yes name=docker-ce state=latest
    - name: Install Docker Module for Python
      pip:
        name: docker
```

A **playbook** is something like that.

An excerpt of an Ansible script to install Docker and Docker bindings for Python on Ubuntu, from: <https://github.com/do-community/ansible-playbooks>

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

Be careful though: infrastructure code is *code*!

The Seven Sins: Security Smells in Infrastructure as Code Scripts

Akond Rahman, Chris Parnin, and Laurie Williams
North Carolina State University, Raleigh, North Carolina
Email: akond.rahman.buet@gmail.com, cjparmin@ncsu.edu, lawilli3@ncsu.edu

Abstract—Practitioners use infrastructure as code (IaC) scripts to provision servers and development environments. While developing IaC scripts, practitioners may inadvertently introduce security smells. Security smells are recurring coding patterns that are indicative of security weakness and can potentially lead to security breaches. *The goal of this paper is to help practitioners avoid insecure coding practices while developing infrastructure as code (IaC) scripts through an empirical study of security smells in IaC scripts.*

We apply qualitative analysis on 1,726 IaC scripts to identify seven security smells. Next, we implement and validate a static analysis tool called Security Linter for Infrastructure as Code scripts (SLIC) to identify the occurrence of each smell in 15,232 IaC scripts collected from 293 open source repositories. We identify 21,201 occurrences of security smells that include 1,326 occurrences of hard-coded passwords. We submitted bug reports for 1,000 randomly-selected security smell occurrences. We obtain 212 responses to these bug reports, of which 148 occurrences were accepted by the development teams to be fixed. We observe security smells can have a long lifetime, e.g., a hard-coded secret can persist for as long as 98 months, with a median lifetime of 20 months.

Index Terms—devops, devsecops, empirical study, infrastructure as code, puppet, security, smell, static analysis

I. INTRODUCTION

Infrastructure as code (IaC) scripts help practitioners to provision and configure their development environment and servers at scale [1]. IaC scripts are also known as configuration scripts [2] [1] or configuration as code scripts [1] [3]. Commercial IaC tool vendors, such as Chef ¹ and Puppet [4], provide programming syntax and libraries so that programmers can specify configuration and dependency information as scripts.

Fortune 500 companies ², such as Intercontinental Exchange (ICE) ³, use IaC scripts to maintain their development environments. For example, ICE, which runs millions of financial transactions daily ⁴, maintains 75% of its 20,000 servers using IaC scripts [5]. The use of IaC scripts has helped ICE decrease the time needed to provision development environments from 1~2 days to 21 minutes [5].

However, IaC scripts can be susceptible to security weakness. Let us consider Figure 1 as an example. In Figure 1, we present a Puppet code snippet extracted from the ‘aeolus-configure’ open source software (OSS) repository ⁵. In this

code snippet, we observe a hard-coded password using the ‘password’ attribute. A hard-coded string ‘v23zj59an’ is assigned as password for user ‘aeolus’. Hard-coded passwords in software artifacts is considered as a software security weakness (‘CWE-798: Use of Hard-coded Credentials’) by Common Weakness Enumerator (CWE) [6]. According to CWE [6], “If hard-coded passwords are used, it is almost certain that malicious users will gain access to the account in question”.

IaC scripts similar to Figure 1, which contain hard-coded credentials or other *security smells*, can be susceptible to security breaches. Security smells are recurring coding patterns that are indicative of security weakness. A security smell does not always lead to a security breach, but deserves attention and inspection. Existence and persistence of these smells in IaC scripts leave the possibility of another programmer using these smelly scripts, potentially propagating use of insecure coding practices. We hypothesize through systematic empirical analysis, we can identify security smells and the prevalence of the identified security smells.

The goal of this paper is to help practitioners avoid insecure coding practices while developing infrastructure as code (IaC) scripts through an empirical study of security smells in IaC scripts.

We answer the following research questions:

- **RQ1:** What security smells occur in infrastructure as code scripts? (Section III)
- **RQ2:** How frequently do security smells occur in infrastructure as code scripts? (Section VI)
- **RQ3:** What is the lifetime of the identified security smell occurrences for infrastructure as code scripts? (Section VI)
- **RQ4:** How do practitioners perceive the identified security smell occurrences? (Section VI)

We answer our research questions by analyzing IaC scripts collected from OSS repositories. We apply qualitative analysis [7] on 1,726 scripts to determine security smells. Next, we construct a static analysis tool called Security Linter for Infrastructure as Code scripts (SLIC) to automatically identify the occurrence of these security smells in 15,232 IaC scripts collected by mining 293 OSS repositories from four sources: Mozilla ⁶, Openstack ⁷, Wikimedia Commons ⁸, and GitHub ⁹.

¹<https://www.chef.io/chef/>

²<http://fortune.com/fortune500/list/>

³<https://www.theice.com/index>

⁴https://www.theice.com/publicdocs/ICE_at_a_glance.pdf

⁵<https://github.com/aeolusproject/aeolus-configure>

⁶<https://hg.mozilla.org/>

⁷<https://git.openstack.org/cgit>

⁸<https://commons.wikimedia.org/>

⁹<https://github.com/>

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

Be careful though: infrastructure code is *code*!

The Sec

Abstract—Practiti
to provision servers
veloping IaC scripts
security smells. Secu
are indicative of se
security breaches. T
avoid insecure codin
code (IaC) scripts th
IaC scripts.
We apply qualita
seven security smell
analysis tool called
scripts (SLIC) to ide
IaC scripts collecte
identify 21,201 occu
occurrences of hard
for 1,000 randomly-s
212 responses to th
were accepted by th
security smells can l
can persist for as lo
20 months.
Index Terms—dev
ture as code, puppe

Infrastructure as
provision and conf
servers at scale [1].
scripts [2] [1] or co
cial IaC tool vendo
programming synta
specify configurati
Fortune 500 com
(ICE) ³, use IaC s
ronments. For exan
transactions daily ⁴.
IaC scripts [5]. The
the time needed to
1~2 days to 21 mi
However, IaC sc
ness. Let us consid
we present a Puppe
configure' open so

¹<https://www.chef.io/>
²<http://fortune.com/fo>
³<https://www.theice.co>
⁴<https://www.theice.co>
⁵<https://github.com/ae>

Information and Software Technology 137 (2021) 106593

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

The do's and don'ts of infrastructure code: A systematic gray literature review

Indika Kumara ^a, Martín Garriga ^a, Angel Urbano Romeu ^a, Dario Di Nucci ^{a,*},
Fabio Palomba ^b, Damian Andrew Tamburri ^a, Willem-Jan van den Heuvel ^a

^a Jheronimus Academy of Data Science, The Netherlands
^b University of Salerno, Italy

ARTICLE INFO

Keywords:
Infrastructure-as-code
DevOps
Gray literature review

ABSTRACT

Context: Infrastructure-as-code (IaC) is the DevOps tactic of managing and provisioning software infrastruc-
tures through machine-readable definition files, rather than manual hardware configuration or interactive
configuration tools.
Objective: From a maintenance and evolution perspective, the topic has picked the interest of practitioners
and academics alike, given the relative scarcity of supporting patterns and practices in the academic literature.
At the same time, a considerable amount of gray literature exists on IaC. Thus we aim to characterize IaC and
compile a catalog of best and bad practices for widely used IaC languages, all using gray literature materials.
Method: In this paper, we systematically analyze the industrial gray literature on IaC, such as blog posts,
tutorials, white papers using qualitative analysis techniques.
Results: We proposed a definition for IaC and distilled a broad catalog summarized in a taxonomy consisting
of 10 and 4 primary categories for best practices and bad practices, respectively, both language-agnostic
and language-specific ones, for three IaC languages, namely Ansible, Puppet, and Chef. The practices reflect
implementation issues, design issues, and the violation of/adherence to the essential principles of IaC.
Conclusion: Our findings reveal critical insights concerning the top languages as well as the best practices
adopted by practitioners to address (some of) those challenges. We evidence that the field of development and
maintenance IaC is in its infancy and deserves further attention.

1. Introduction

The current information technology (IT) market is increasingly
focused on "need for speed": speed in deployment, faster release-cycles,
speed in recovery, and more. This need is reflected in DevOps, a
family of techniques that shorten the software development cycle and
intermix software development activities with IT operations [1,2]. As
part of the DevOps, infrastructure-as-code (IaC) [3] promotes managing
the knowledge and experience inside reusable scripts of infrastructure
code, instead of traditionally reserving it for the manual-intensive labor
of system administrators, which is typically slow, time-consuming,
effort-prone, and often even error-prone.

IaC represents a widely adopted practice [3–5]. However, little
is known concerning its code maintenance, evolution, and continu-
ous improvement in academic literature, despite increasing traction
in most if not all domains of society and industry: from Network-
Function Virtualisation (NFV) [6] to Software-Defined Everything [7]
and more [8].

However, little academic literature exists on infrastructure code
since research on IaC is still in its early phases. At the same time,
companies are working day-by-day on their infrastructure automation,
as also witnessed by the considerable amount of gray literature on the
topic. Hence, we can observe a gap between academic research and
industry practices, mainly to figure out the technical, operational, and
theoretical underpinnings and the best and bad practices when developing
IaC in the most popular languages.

This paper aims at addressing this gap with a systematic gray litera-
ture review. We shed light on the state of the practice in the adoption of
IaC by analyzing 67 high-quality sources and the fundamental software
engineering challenges in the field. In particular, we investigate: (1)
how the industrial researchers and practitioners characterize IaC, and
(2) the best/bad practices during general (i.e., language-agnostic) and
language-specific (e.g., Puppet, Chef, Ansible) IaC development.

Specifically, we derived a more rigorous definition for infrastructure
code. We identified a taxonomy consisting of 10 primary categories for
best practices and 4 for bad practices, with all practices reflecting key

* Corresponding author.
E-mail addresses: i.p.k.weerasingha.dewage@tue.nl (I. Kumara), m.garriga@uvt.nl (M. Garriga), a.urbanoromeu@uvt.nl (A.U. Romeu), d.dinucci@uvt.nl
(D. Di Nucci), f.palomba@unisa.it (F. Palomba), d.a.tamburri@tue.nl (D.A. Tamburri), w.j.a.m.vdnheuvel@uvt.nl (W.-J. van den Heuvel).

<https://doi.org/10.1016/j.infsof.2021.106593>
Received 11 July 2020; Received in revised form 5 March 2021; Accepted 2 April 2021
Available online 29 April 2021
0950-5849/© 2021 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

AI-Enabled Systems in Operation: Infrastructure-as-Code

Be careful though: infrastructure code is code!

The Se

Abstract—Practiti to provision servers veloping IaC scripts security smells. Secu are indicative of se security breaches. 7 avoid insecure codin code (IaC) scripts th IaC scripts.

We apply qualita seven security smell analysis tool called scripts (SLIC) to ide IaC scripts collecte identify 21,201 occu occurrences of hard for 1,000 randomly-s 212 responses to th were accepted by th security smells can l can persist for as lo 20 months.

Index Terms—dev ture as code, puppe



The do's and

Indika Kumara
Fabio Palomba

^aJheronimus Academy of
^bUniversity of Salerno, Ita

ARTICLE 11

Keywords:
Infrastructure-as-code
DevOps
Gray literature review

Within-Project Defect Prediction of Infrastructure-as-Code Using Product and Process Metrics

Stefano Dalla Palma ¹, Dario Di Nucci ¹, Fabio Palomba ¹, and Damian A. Tamburri ¹

Abstract—Infrastructure-as-code (IaC) is the DevOps practice enabling management and provisioning of infrastructure through the definition of machine-readable files, hereinafter referred to as *IaC scripts*. Similarly to other source code artefacts, these files may contain defects that can preclude their correct functioning. In this paper, we aim at assessing the role of *product* and *process* metrics when predicting defective IaC scripts. We propose a fully integrated machine-learning framework for IaC Defect Prediction, that allows for repository crawling, metrics collection, model building, and evaluation. To evaluate it, we analyzed 104 projects and employed five machine-learning classifiers to compare their performance in flagging suspicious defective IaC scripts. The key results of the study report RANDOM FOREST as the best-performing model, with a median AUC-PR of 0.93 and MCC of 0.80. Furthermore, at least for the collected projects, product metrics identify defective IaC scripts more accurately than process metrics. Our findings put a baseline for investigating IaC Defect Prediction and the relationship between the product and process metrics, and IaC scripts' quality.

Index Terms—Infrastructure-as-code, defect prediction, empirical software engineering

1 INTRODUCTION

MODERN software relies on the use of automation at both development and operations levels, an engineering strategy known as DevOps [1]; the software code driving such automation is collectively known as Infrastructure-as-Code (IaC). Infrastructure-as-Code “promotes managing the knowledge and experience inside reusable scripts of infrastructure code instead of traditionally reserving it for the manual-intensive labor of system administrators, typically slow, time-consuming, effort-heavy, and often even error-prone” [2].

IaC tactic emphasizes consistent, repeatable routines for provisioning, deploying, and orchestrating systems. Infrastructure managers can build systems that automatically change the infrastructure. Specifically, they can develop, test, and deploy these systems using the same engineering practices and tooling proven effective for application development, such as version control systems and automated testing. On the one hand, this requires an infrastructure to cope with continuous and rapid changes common in established software development practices, like Agile development. On the other hand, this opens the door to exploit development practices such as test-driven development, continuous integration, and continuous delivery.

Although IT infrastructures are constantly evolving and growing in size and complexity, little is known concerning how to maintain best, speedily evolve, and continuously improve infrastructure code, and yet it is picking up more and more traction in different domains [3]. This is problematic for organizations where software is essential. Infrastructure failures are even more demanding in environments where IT systems are more than just business-critical and where there is no tolerance for downtime. For example, Amazon’s systems handle hundreds of millions of dollars in transactions every day. In that context, only in 2012, the estimated average cost of one-minute service downtime for Amazon alone was \$66,000,¹ even after extensive manual and semi-automatic service continuity practices such as service hot stand-by [4], or elastic provisioning [5].

Software Defect Prediction [6] supports the software development life-cycle testing process by identifying the parts of the system that are failure-prone and require extensive testing. As shown by previous research in the field, like any other source code artifact, infrastructure configuration management scripts can be failure-prone [7], [8], [9]. The effective prediction of failure-prone IaC scripts may enable organizations embracing the DevOps methodology to focus on such critical scripts during Quality Assurance activities and allocate effort and resources more efficiently.

Therefore, this work aims to help software practitioners prioritize their inspection efforts for IaC scripts by proposing prediction models of failure-prone IaC scripts and investigating the role of product and process metrics for their prediction.

¹ <https://www.chef.io/>

² <http://fortune.com/fo>

³ <https://www.theice.co>

⁴ <https://www.theice.co>

⁵ <https://github.com/ae>

* Corresponding au
E-mail addresses:
(D. Di Nucci), fpalomb@unisa.it

<https://doi.org/10.101>

Received 11 July 2024

Available online 29 Ap

0950-5849/© 2021 Th

Manuscript received 29 May 2020; revised 7 Jan. 2021; accepted 11 Jan. 2021.

Date of publication 13 Jan. 2021; date of current version 14 June 2022.

(Corresponding author: Stefano Dalla Palma.)

Recommended for acceptance by L. Mariani.

Digital Object Identifier no. 10.1109/TSE.2021.3051492

1. As determined by Forbes based on Amazon’s 2012 net sales.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see <https://creativecommons.org/licenses/by/4.0/>

AI-Enabled Systems in Operation: Infrastructure-as-Code

Be careful though: infrastructure code is code!

The Se

Abstract—Practiti to provision servers veloping IaC scripts security smells. Secu are indicative of se security breaches. 7 avoid insecure codin code (IaC) scripts th IaC scripts.

We apply qualita seven security smell analysis tool called scripts (SLIC) to ide IaC scripts collecte identify 21,201 occu occurrences of hard for 1,000 randomly-s 212 responses to th were accepted by th security smells can l can persist for as lo 20 months.

Index Terms—dev ture as code, puppe

Infrastructure as provision and conf servers at scale [1]. scripts [2] [1] or co cial IaC tool vendo programming synta specify configurati Fortune 500 com (ICE) ³, use IaC s onments. For exan transactions daily ⁴, IaC scripts [5]. The the time needed to 1~2 days to 21 mi

However, IaC sc ness. Let us consid we present a Puppe configure' open so

¹ <https://www.chef.io/>
² <http://fortune.com/fo>
³ <https://www.theice.co>
⁴ <https://www.theice.co>
⁵ <https://github.com/ae>


ELSEVIER

The do's and
Indika Kumara
Fabio Palomba

^a Jheronimus Academy of
^b University of Salerno, Ita

ARTICLE 11

Keywords:
Infrastructure-as-code
DevOps
Gray literature review

Within-Project Defect Prediction of Infrastructure-as-Code Using Product and Process Metrics

Stefano Dalla Palma ^a, Dario Di Nucci ^b, Fabio Palomba ^b, and Damian A. Tamburri ^b

Abstract—Infrastructure-as-code (IaC) is the DevOps practice enabling management and provisioning of infrastructure through the definition of machine-readable files, hereinafter referred to as *IaC scripts*. Similarly to other source code artefacts, these files may contain defects that can preclude their correct functioning. In this paper, we aim at assessing the role of *product* and *process* metrics when predicting defective IaC scripts. We propose a fully integrated machine-learning framework for IaC Defect Prediction, that allows for repository crawling, metrics collection, model building, and evaluation. To evaluate it, we analyzed 104 projects and employed five machine-learning classifiers to compare their performance in flagging suspicious defective IaC scripts. The key results of the study report RANDOM FOREST as the best-performing model, with a median AUC-PR of 0.93 and MCC of 0.80. Furthermore, at least for the collected projects, product metrics identify defective IaC scripts more accurately than process metrics. Our findings put a baseline for investigating IaC Defect Prediction and the relationship between the product and process metrics, and IaC scripts' quality.

Index Terms—Infrastructure-as-code, defect prediction, empirical software engineering

1 INTRODUCTION

MODERN software relies on the use of automation at both development and operations levels, an engineering strategy known as DevOps [1]; the software code driving such automation is collectively known as Infrastructure-as-Code (IaC). Infrastructure-as-Code “promotes managing the knowledge and experience inside reusable scripts of infrastructure code instead of traditionally reserving it for the manual-intensive labor of system administrators, typically slow, time-consuming, effort-heavy, and often even error-prone” [2].

IaC tactic emphasizes consistent, repeatable routines for provisioning, deploying, and orchestrating systems. Infrastructure managers can build systems that automatically change the infrastructure. Specifically, they can develop, test, and deploy these systems using the same engineering practices and tooling proven effective for application development, such as version control systems and automated testing. On the one hand, this requires an infrastructure to cope with continuous and rapid changes common in established software development practices, like Agile development. On the other hand, this opens the door to exploit development

practices such as test-driven development, continuous integration, and continuous delivery.

Although IT infrastructures are constantly evolving and growing in size and complexity, little is known concerning how to maintain best, speedily evolve, and continuously improve infrastructure code, and yet it is picking up more and more traction in different domains [3]. This is problematic for organizations where software is essential. Infrastructure failures are even more demanding in environments where IT systems are more than just business-critical and where there is no tolerance for downtime. For example, Amazon’s systems handle hundreds of millions of dollars in transactions every day. In that context, only in 2012, the estimated average cost of one-minute service downtime for Amazon alone was \$66,000,¹ even after extensive manual and semi-automatic service continuity practices such as service hot stand-by [4], or elastic provisioning [5].

Software Defect Prediction [6] supports the software development life-cycle testing process by identifying the parts of the system that are failure-prone and require extensive testing. As shown by previous research in the field, like any other source code artifact, infrastructure configuration management scripts can be failure-prone [7], [8], [9]. The effective prediction of failure-prone IaC scripts may enable organizations embracing the DevOps methodology to focus on such critical scripts during Quality Assurance activities and allocate effort and resources more efficiently.

Therefore, this work aims to help software practitioners prioritize their inspection efforts for IaC scripts by proposing prediction models of failure-prone IaC scripts and investigating the role of product and process metrics for their prediction.

^a Stefano Dalla Palma and Dario Di Nucci are with the Jheronimus Academy of Data Science, Tilburg University, 5037 AB Tilburg, The Netherlands. E-mail: s.dallapalma@uvt.nl, d.dinucci@uvt.nl.

^b Fabio Palomba is with the Software Engineering (SeSa) Lab, University of Salerno, 84084 Fisciano, Italy. E-mail: fpalomba@unisa.it.

^c Damian A. Tamburri is with the Jheronimus Academy of Data Science, Eindhoven University of Technology, 5612 AZ Eindhoven, The Netherlands. E-mail: d.a.tamburri@tue.nl.

Manuscript received 29 May 2020; revised 7 Jan. 2021; accepted 11 Jan. 2021.
Date of publication 13 Jan. 2021; date of current version 14 June 2022.
(Corresponding author: Stefano Dalla Palma.)
Recommended for acceptance by L. Mariani.
Digital Object Identifier no. 10.1109/TSE.2021.3051492

1. As determined by Forbes based on Amazon’s 2012 net sales.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see <https://creativecommons.org/licenses/by/4.0/>

lab

S e S e S e

SOFTWARE ENGINEERING
SALERNO

Jade lab

We rock science with love

 UNIVERSITÀ
DEGLI STUDI
DI SALERNO

JADS
Jheronimus
Academy
of Data Science

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Orchestration

To summarize, up to this step we would have:

- 1 An AI component, which is integrated within a larger software system through the use of software architectures like, for instance, microservices.
- 2 A mechanism to embed microservices within containers, so that all dependencies required by the AI component (and non-AI components) are packaged together in a self-contained and portable environment, ensuring consistent execution across different infrastructure setups.
- 3 A mechanism to deploy containers using Infrastructure-as-Code, which ensures that containerized applications are provisioned, configured, and managed declaratively through automated scripts.

Just one question is pending: *how to efficiently manage and scale these containers?*

We may have different available hardware, with slower or faster CPUs, with more or less memory, and with or without GPUs. In such settings, after virtualizing all the components of a system, operators still need to decide how many instances to start for each container, which instance to run on which hardware, and how to route requests.

The cloud-computing community has developed much infrastructure to ease and automate such tasks, which is collectively known as **orchestration** - we are not going into that, but a well-known example is Kubernetes.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

After architecting and deployment, we have an AI-enabled system ready to be receiving external inputs and making predictions. This phase is called *operation*.

Operation. The tasks, infrastructure, and processes involved in deploying, running, and updating a software system and the hardware it runs on in production.

Operations is often considered as a **distinct activity** from the development of the software and is often performed by other team members with dedicated expertise.

DevOps and **MLOps** emerged as concepts and infrastructure to support collaboration between developers, data scientists, and operators toward better operations.

Compared to traditional software, introducing AI raises additional challenges during operations, such as (a) ensuring that model training and model inference operate well and (b) moving and processing very large amounts of data.

In addition, AI-enabled systems often have demanding requirements for updates and experimentation due to continuous experimentation and drifting data.

When software projects with AI components reach a certain scale or complexity, organizations should consider bringing in operations specialists, who may have role titles such as **system administrators**, **release engineers**, **site reliability engineers**, **DevOps engineers**, **MLOps engineers**, **data engineers**, and **AI engineers**.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation

Service level objectives are negotiated and renegotiated between operators and other stakeholders, reasoning about constraints, priorities, and trade-offs as usual.

For example, the increased infrastructure cost to increase availability from 99.9 to 99.99 percent might simply outweigh the benefit; even lower availability might be acceptable for a service if it allows the operation team to focus on more important goals.

In more formal arrangements, service-level objectives may be codified as contracts called **service-level agreements**, which typically include a description of consequences and penalties if the objectives are not met.

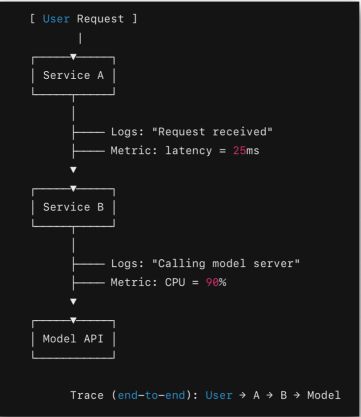
Service-level agreement. This is a formal contract between a *service provider* and a *client* that defines the *expected level of service*, including key performance metrics such as availability, response time, throughput, and error rates. It also *specifies responsibilities, penalties for non-compliance, and any compensation mechanisms* in case of service failures.

Nowadays, SLAs are *widely used across industries* to ensure service reliability, maintain accountability, and enhance customer satisfaction in both *business-to-business (B2B)* and *business-to-customer (B2C)* interactions.

For example, consider when you subscribe to a mobile phone plan: you formally agree the terms of service and the contract is the basis for the mutual relationship.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: The Three Pillars of Observability



Altogether, metrics, logs, and traces offer an instrument to continuously monitor and analyze how services work and how they interact.

Model retraining, model rollback, and alerting are decisions typically taken by analyzing observability data, such as feature and prediction drift, degradation in model accuracy or confidence, and system-level metrics like latency and error rates.

At the same time, observability data allow auditing the behavior of AI-enabled systems over time, helping to ensure compliance with regulations, detect unexpected behavior, and trace back the causes of incorrect/unfair predictions.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

Observability is strictly connected to the concept of automating the deployment of AI-enabled systems. Indeed, most software systems must be updated frequently to fix bugs and to deploy new features. Long gone are the days where a new software version is released every three years; today, many organizations release and deploy updates **weekly** or even **multiple times a day**.

Deploying software and software updates is a **core responsibility of operators**, and deployment is often a source of frustration. Unscheduled patches to fix an urgent problem live in production are an operator's nightmare.

Operators often invest substantial effort in automating deployments. By eliminating manual steps, they avoid mistakes and inconsistencies, increase the speed with which updates can be deployed, and support rolling back problematic updates.

Most organizations start automation with a *continuous integration* service.

Continuous Integration. The process of automatically building and testing every code change integrated into the version control system.

Continuous integration ensures that each version can be built independently (not just on the developer's machine), pass all tests, and pass all other automated quality checks (e.g., static analysis).

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Automating Deployments

Integrating CI with continuous delivery/deployment (CD) systems allows teams to automate deployment processes and validate software in production-like environments, enhancing reliability and reducing time-to-market.

Continuous Delivery. The process of automatically staging code changes for release, ensuring that software can be deployed to production at any time with minimal manual intervention.

In the simplest case, handwritten scripts upload binaries or models to a server, but often more sophisticated steps are involved, e.g., packaging the artifacts in a *container*, uploading the container to a cloud service, launching one or more instances of the container, adjusting network routing, and then shutting down the previous containers.

A **container** is a lightweight, portable, software package that includes everything needed to run an application—such as code, runtime, system tools, libraries, and settings—ensuring consistent execution across different computing environments.



Docker is a well-known example: it allows developers to package applications and their dependencies into standardized containers, which can run reliably on any environment that supports the Docker engine—whether it's a developer's laptop, a testing server, or a production system in the cloud.

Software Engineering for Artificial Intelligence: AI-Enabled Systems in Operation

AI-Enabled Systems in Operation: Infrastructure-as-Code

While virtualization guarantees that operators should not worry about the underlying code requirements, they still require setting up and managing virtual machines, including operating systems, resource allocation, and updates.

To avoid repetitive manual system configuration work and potential inconsistencies from manual changes, there is a strong push toward automating configuration changes.

That is, rather than remotely accessing a system to install a binary and adding a line to a configuration file, an operator will write a script to perform these changes and then run the script on the target machine.

Please notice that even if the configuration is applied only to a single machine, having a script will enable setting up the machine again in the future, without having to remember all manual steps done in the past. In addition, it is now possible to apply the same steps to many machines to configure them consistently, e.g., to ensure that they all run the same firewall configuration.

Infrastructure-as-code. The practice of managing and provisioning computing infrastructure through machine-readable configuration files, rather than manual processes, enabling automation, consistency, and repeatability.

The scripts and all configuration values they set are usually tracked in version control, just like any source code in the project.



Software Engineering for Artificial Intelligence

AI-Enabled Systems in Operation

